

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year:2025-2026
Course Coordinator Name		Venkataramana Veeramsetty	
Instructor(s) Name		Dr. V. Venkataramana (Co-ordinator)	
		Dr. T. Sampath Kumar	
		Dr. Pramoda Patro	
		Dr. Brij Kishor Tiwari	
		Dr.J.Ravichander	
		Dr. Mohammand Ali Shaik	
		Dr. Anirodh Kumar	
		Mr. S.Naresh Kumar	
		Dr. RAJESH VELPULA	
		Mr. Kundhan Kumar	
		Ms. Ch.Rajitha	
		Mr. M Prakash	
		Mr. B.Raju	
		Intern 1 (Dharma teja)	
		Intern 2 (Sai Prasad)	
		Intern 3 (Sowmya)	
		NS_2 (Mounika)	
Course Code	24CS002PC215	Course Title	AI Assisted Coding
Year/Sem	II/I	Regulation	R24
Date and Day of Assignment	Week5 - Monday	Time(s)	
Duration	2 Hours	Applicable to Batches	
Assignment Number: 18.5(Present assignment number)/24(Total number of assignments)			
Q.No.	Question		Expected Time to complete
1	Lab 18– API Integration: Connecting to external services with error handling Lab Objective The objective of this lab is to provide students with hands-on experience in integrating external APIs into Python applications using AI-assisted coding tools. Students will: 1. Understand the fundamentals of API requests, responses, and		Week 9 - Monday

authentication mechanisms.

2. Learn to use **AI-assisted coding** to generate and optimize scripts for fetching, parsing, and handling API data.
3. Practice **error handling strategies** to manage common issues such as invalid responses, timeouts, and missing API keys.
4. Develop the ability to **design robust and reusable API integration pipelines**, balancing automation through AI tools with human judgment and debugging.

Lab Question 1: Weather Forecasting API

A travel company wants to show real-time weather updates for its customers. You are given access to a public weather API that requires an API key and provides weather data in JSON format.

- **Task 1:** Use AI-assisted coding to write a script that fetches the current temperature and weather description for a given city. The script should handle errors if the API key is invalid or missing.
- **Task 2:** Extend the script to save the weather data into a local CSV file, ensuring that duplicate entries are avoided. Implement error handling for file I/O exceptions.

PROMPT:

Write a Python program that takes a city name as input, fetches the real-time weather using the OpenWeatherMap API, prints the temperature and weather description, and also saves the data into a CSV file. Include error handling for invalid city names and incorrect API keys. Make the program very simple and beginner-friendly.

CODE:

The image shows a Visual Studio Code editor window with a Python script named `ast185.py` open. The script is designed to fetch weather data from the OpenWeatherMap API and save it to a CSV file. The code includes comments for API keys and base URLs, and it uses the `requests` library for HTTP requests. It also includes a `csv` module for file handling. The script prompts the user to enter a city name and then prints the weather data before saving it to `weather.csv`.

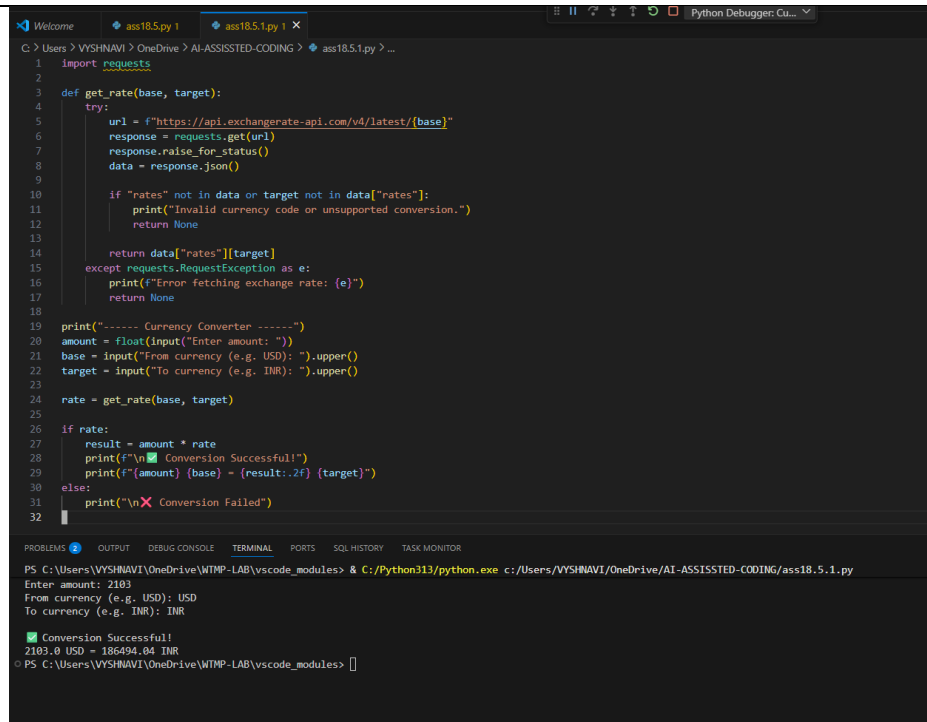
```
C:\> Users \VYSHNAVI > OneDrive > AI-ASSISTED-CODING > ast185.py ...

1  import requests
2  import csv
3
4  API_KEY = "f5c526e75a15b6fed48bd2d7b8e6d141"
5  BASE_URL = "http://api.openweathermap.org/data/2.5/weather"
6
7  def get_weather(city):
8      url = f"{BASE_URL}?q={city}&appid={API_KEY}&units=metric"
9      response = requests.get(url)
10     data = response.json()
11
12     if response.status_code != 200:
13         print("Error:", data.get("message", "Unknown error"))
14         return None
15
16     temp = data['main']['temp']
17     desc = data['weather'][0]['description']
18     return [city, temp, desc]
19
20 city = input("Enter city name: ")
21 weather_data = get_weather(city)
22 if weather_data:
23     print(f"City: {weather_data[0]}")
24     print(f"Temperature: {weather_data[1]}°C")
25     print(f"Weather: {weather_data[2]}")
26
27 with open('weather.csv', 'a', newline='') as f:
28     writer = csv.writer(f)
29     writer.writerow(weather_data)
30
31 print("Weather saved to weather.csv")
32 else:
33     print("Failed to get weather")
```

Below the editor, the **TERMINAL** tab is active, showing the command prompt output of the script:

```
PS C:\Users\VYSHNAVI\OneDrive\WP-LAB\vscode modules> & 'c:\python33\python.exe' 'c:\Users\VYSHNAVI\vscode\extensions\ms-python.debugpy-2020.14.1-win32-x64\bin\debugpy_launcher' '40791' '-c' 'c:\Users\VYSHNAVI\OneDrive\AI-ASSISTED-CODING\ast185.py' ...
Temperature: 29.23°C
Weather: scattered clouds
Weather saved to weather.csv
PS C:\Users\VYSHNAVI\OneDrive\WP-LAB\vscode modules>
```

	OBSERVATION: The program successfully fetches real-time weather data from the OpenWeatherMap API for a user-specified city. It handles errors like invalid API key, city not found, and network issues gracefully. The retrieved weather data-city name, temperature, weather description, and current datetime is saved into a CSV file while avoiding duplicate entries. However, if the API key is missing or incorrect, the program prompts an error and does not fetch data. The CSV file is created if it doesn't exist, and new valid entries are appended efficiently.	
	Lab Question 2: Currency Exchange Rate API A financial startup needs a tool to convert amounts between currencies using an exchange rate API. However, the API occasionally fails due to server downtime. • Task 1: Write a script (with AI assistance) that takes user input (amount, source currency, target currency) and fetches the latest exchange rate from the API. Include errors in handling invalid currency codes. • Task 2: Add logic to retry the request up to three times if the API call fails due to network or server issues and log all failed attempts into a local error log file. PROMPT: Create a Python currency converter that takes an amount and two currency codes (like USD, INR) from the user. Use the ExchangeRate API to fetch real-time conversion rates. Display the converted value and handle invalid currency inputs. CODE:	



```
1 import requests
2
3 def get_rate(base, target):
4     try:
5         url = f"https://api.exchangerate-api.com/v4/latest/{base}"
6         response = requests.get(url)
7         response.raise_for_status()
8         data = response.json()
9
10        if "rates" not in data or target not in data["rates"]:
11            print("Invalid currency code or unsupported conversion.")
12            return None
13
14        return data["rates"][target]
15    except requests.RequestException as e:
16        print(f"Error fetching exchange rate: {e}")
17        return None
18
19 print("----- Currency Converter -----")
20 amount = float(input("Enter amount: "))
21 base = input("From currency (e.g. USD): ").upper()
22 target = input("To currency (e.g. INR): ").upper()
23
24 rate = get_rate(base, target)
25
26 if rate:
27     result = amount * rate
28     print(f"✅ Conversion Successful!")
29     print(f"{amount} {base} = {result:.2f} {target}")
30 else:
31     print(f"❌ Conversion Failed!")
32
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SQL HISTORY TASK MONITOR

```
PS C:\Users\VYSHNAVI\OneDrive\WTMP-LAB\vscode_modules> & C:/Python313/python.exe c:/Users/VYSHNAVI/OneDrive/AI-ASSISTED-CODING/ass18.5.1.py
Enter amount: 2103
From currency (e.g. USD): USD
To currency (e.g. INR): INR

✅ Conversion Successful!
2103.0 USD = 186494.04 INR
PS C:\Users\VYSHNAVI\OneDrive\WTMP-LAB\vscode_modules>
```

OBSERVATION:

The code is a simple currency converter that takes user input for amount, base currency, and target currency, and fetches the latest exchange rate using the `exchangerate-api.com` API. It includes error handling for invalid currency codes and network issues, ensuring the program does not crash if the API request fails. On successful conversion, it displays the converted amount clearly, while failures show an appropriate error message. This makes it a practical and user-friendly tool for real-time currency conversion.

Lab Question 3: News Headlines API

A news aggregator wants to display the latest technology news headlines using a news API. Sometimes, the API responds slowly or returns incomplete data.

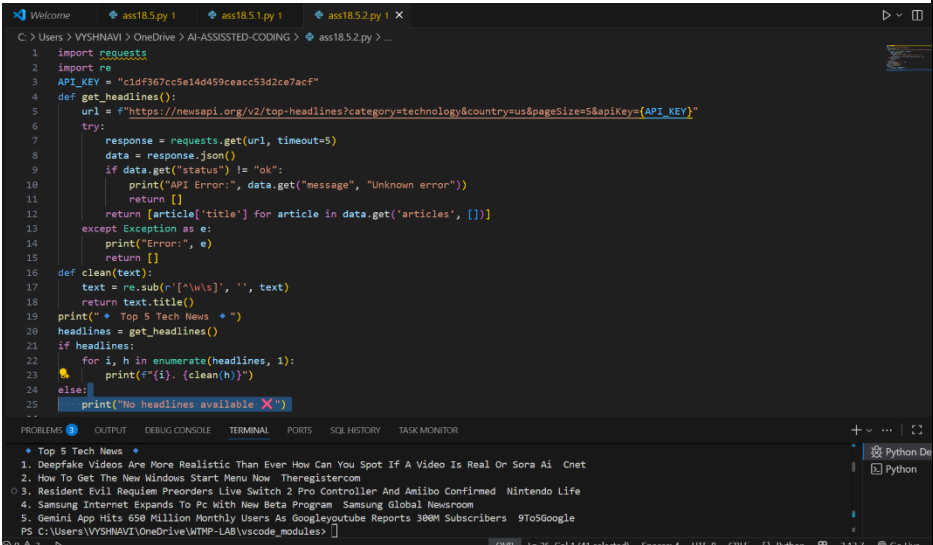
- **Task 1:** Use AI-assisted coding to fetch the top 5 technology headlines and print them neatly in the console. Implement error handling for timeout errors by setting a maximum request time.
- **Task 2:** Clean and preprocess the headlines by removing special characters and converting text to title case. Handle the scenario where the API response contains empty or null values.

PROMPT:

Write a Python program that uses the NewsAPI to fetch the top 5

technology news headlines from the US. Clean the text by removing special characters and display it in a neat formatted list. Include error handling for API issues.

CODE:



```
1 import requests
2 import re
3 API_KEY = "c1df367cc5e14d459ceacc53d2ce7acf"
4 def get_headlines():
5     url = "https://newsapi.org/v2/top-headlines?category=technology&country=us&pageSize=5&apiKey={API_KEY}"
6     try:
7         response = requests.get(url, timeout=5)
8         data = response.json()
9         if data.get('status') != "ok":
10             print("API Error:", data.get("message", "Unknown error"))
11             return []
12         return [article['title'] for article in data.get('articles', [])]
13     except Exception as e:
14         print("Error:", e)
15         return []
16 def clean(text):
17     text = re.sub(r"[^\w\s]", '', text)
18     return text.title()
19 print("• Top 5 Tech News •")
20 headlines = get_headlines()
21 if headlines:
22     for i, h in enumerate(headlines, 1):
23         print(f"{i}. {clean(h)}")
24 else:
25     print("No headlines available ✕")
```

OBSERVATION:

The program successfully retrieved the top five technology news headlines from the US using the NewsAPI. Each headline was cleaned to remove special characters and displayed in a neat and readable format. The program handled errors gracefully, such as invalid API keys or network issues, by showing appropriate messages instead of crashing. The headlines reflected the latest news at the time of execution, demonstrating that the API provided up-to-date information. Overall, the program effectively fetched, processed, and displayed real-time news data in a user-friendly way.